straightforward redirect without having to construct a whole route or controller:

```
Route::redirect('/ep1', '/ep2');
```

The status code can be customized as well for the route:

```
Route::redirect('/ep1', '/ep2', 301);
```

Alternatively, the **`Route::permanentRedirect`** method can be used to get a 301 status code:

```
Route::permanentRedirect('/ep1', '/ep2');
```

View Routes

Use the **`Route::view`** method if your route needs to return only a **`view`**. This technique offers a shortcut so that you do not need to define a whole route or controller, similar to the redirect method.

The view method takes two arguments:

a URI as the first and

a view name as the second.

As an optional third argument, you may also supply an array of data to the view:

```
Route::view('/about', 'about');
Route::view('/about', 'about', ['state' => 'Reader']);
```

The Route List

The **`route:list`** The artisan command can quickly give you a summary of all the routes that your application has created.

You can tell Laravel to show the route middleware by adding the -v option to the command: By default, the route middleware that is assigned to each route will not be displayed in the **`route:list`** output.

```
php artisan route:list -v
```

By adding the **`--except-vendor`** option while running the **`route:list`**, you can also instruct Laravel to conceal any routes.

```
php artisan route:list --except-vendor
```